

BÁO CÁO THỰC NGHIỆM: CHIẾN LƯỢC TỐI ƯU HÓA PROMPT TRONG HỌC TẬP

Người thực hiện: Phùng Trung Dũng

Mã sinh viên: 25020506

Lớp: E-CE5

Khóa: K70

Ngành: Kỹ thuật máy tính

Trường: Đại học Công nghệ (UET) – ĐHQGHN

I. Lựa chọn và phân tích tác vụ học tập

Để tối ưu hóa quá trình học tập, tôi lựa chọn 3 tác vụ sau:

- Tóm tắt tài liệu học thuật (Tác vụ Tổng hợp): Giảm tải khối lượng đọc hiểu các bài báo khoa học dài, trích xuất nhanh các thông số kỹ thuật.
- Giải thích khái niệm phức tạp (Tác vụ Tư duy): Chuyển đổi các kiến thức trừu tượng thành các ví dụ trực quan, dễ hiểu.
- Tạo bộ câu hỏi ôn tập (Tác vụ Kiểm tra): Tự đánh giá mức độ hiểu bài thông qua các câu hỏi đa dạng, từ đó củng cố lỗ hổng kiến thức.

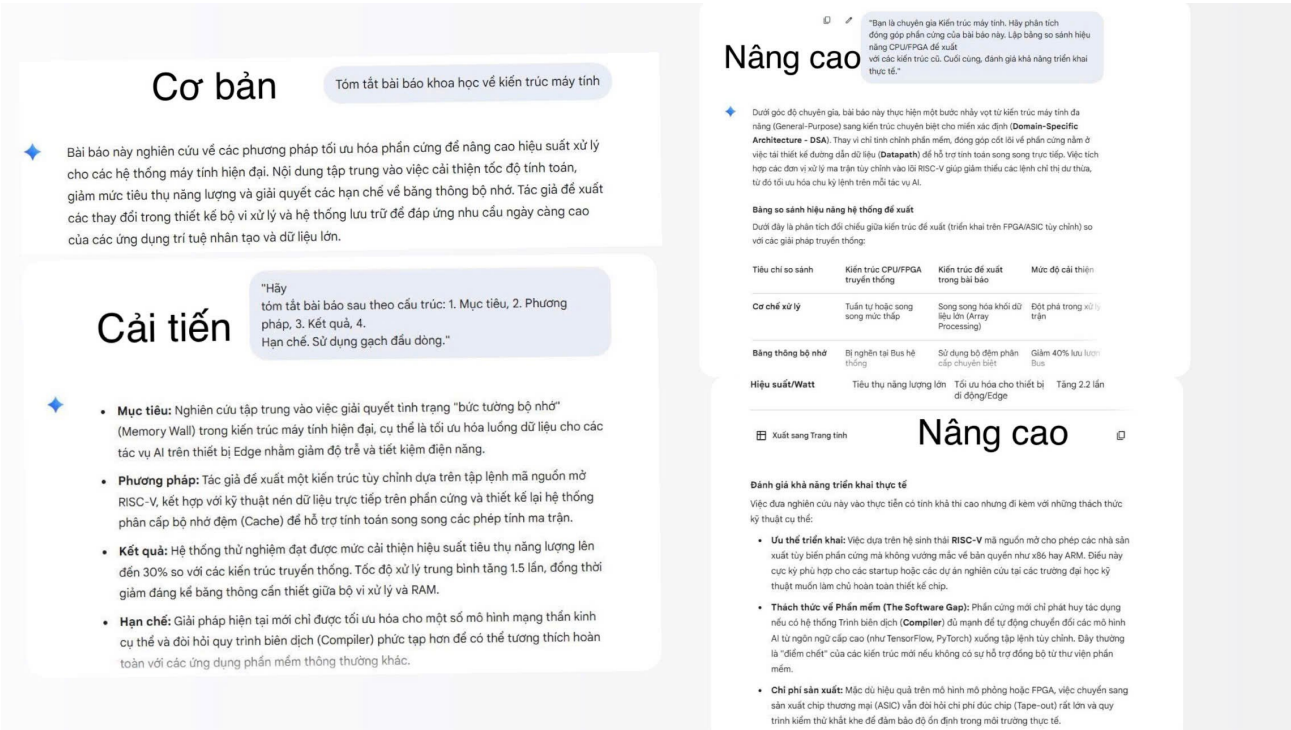
II. Xây dựng và Thử nghiệm các phiên bản Prompt

Tôi thực hiện thực nghiệm trên 3 tác vụ cốt lõi, mỗi tác vụ được vận hành qua 3 cấp độ cấu trúc câu lệnh khác nhau để quan sát sự biến đổi về chất lượng thông tin.

1. Tác vụ 1: Tóm tắt bài báo khoa học về kiến trúc máy tính.

Cấp độ	Nội dung Prompt	Sắc thái kết quả quan sát
Cơ bản	"Tóm tắt bài báo khoa học về kiến trúc máy tính"	Kết quả thường quá ngắn hoặc bỏ lỡ các thông số kỹ thuật quan trọng
Cải tiến	"Hãy tóm tắt bài báo sau theo cấu trúc rõ ràng hơn, thông tin có tính"	Cấu trúc rõ ràng hơn, thông tin có tính

	trúc: 1. Mục tiêu, 2. Phương pháp, 3. Kết quả, 4. Hạn chế. Sử dụng gạch đầu dòng."	phân loại, giúp người đọc nắm bắt nhanh các ý chính của nghiên cứu. Tuy nhiên, kết quả còn thiếu sự liên kết giữa các phần.
Nâng cao	"Bạn là chuyên gia Kiến trúc máy tính. Hãy phân tích đóng góp phần cứng của bài báo này. Lập bảng so sánh hiệu năng CPU/FPGA đề xuất với các kiến trúc cũ. Cuối cùng, đánh giá khả năng triển khai thực tế."	Vai trò chuyên gia (Role) và Yêu cầu bảng (Output format) buộc AI phải so sánh dữ liệu, tránh việc tóm tắt hời hợt. Làm cho người đọc nhận được kiến thức đầy đủ từ bài báo nhưng vẫn tiết kiệm được thời gian và đặc biệt là được phân tích rất dễ nhớ, giúp ích trực tiếp cho việc làm báo cáo khoa học.



Hình 1: So sánh sự dịch chuyển từ văn bản liệt kê sang tư duy phân tích hệ thống.

2. Tác vụ 2: Giải thích khái niệm "Con trỏ trong ngôn ngữ C"

Cấp độ	Nội dung Prompt	Sắc thái kết quả quan sát
Cơ bản	"Giải thích con trỏ trong C là gì?"	Định nghĩa mang tính từ điển, khó hình dung đối với người mới học.
Cải tiến	"Giải thích con trỏ trong C. Cho một ví dụ thực tế và viết đoạn code minh họa đơn giản."	Có sự kết nối giữa lý thuyết và thực hành, giúp dễ hình dung hơn thông qua ví dụ.

Nâng cao	"Sử dụng ẩn dụ 'địa chỉ nhà' để giải thích con trỏ. Thực hiện theo 3 bước (Chain-of-Thought): 1. Khai báo, 2. Gán địa chỉ, 3. Truy xuất. Nhấn mạnh lỗi Segmentation Fault."	Chain-of-Thought (CoT) dẫn dắt tư duy theo logic từ khóa, giúp người học hiểu "luồng" dữ liệu đi qua bộ nhớ, giảm thiểu lỗi lập trình sau này.
----------	---	---

Cơ bản

"Giải thích con trỏ trong C là gì?"

Con trỏ trong ngôn ngữ lập trình C là một loại biến đặc biệt, thay vì lưu trữ các giá trị dữ liệu thông thường như số nguyên hay ký tự, nó dùng để lưu trữ địa chỉ bộ nhớ của một biến khác. Khi bạn biết địa chỉ này, bạn có thể truy cập hoặc thay đổi giá trị của biến đó một cách gián tiếp thông qua con trỏ.

Cải tiến

"Giải thích con trỏ trong C. Cho một ví dụ thực tế và viết đoạn code minh họa đơn giản."

Con trỏ trong C là một biến dùng để lưu trữ **địa chỉ bộ nhớ** của một biến khác thay vì lưu trữ giá trị trực tiếp.

Để dễ hình dung, hãy tưởng tượng một ví dụ thực tế: Bạn có một "ngôi nhà" (biến) chứa "đồ đạc" (giá trị). Thay vì bê cả ngôi nhà đi, bạn chỉ cần ghi lại **địa chỉ nhà** vào một tờ giấy. Tờ giấy đó chính là **con trỏ**. Khi bạn đưa tờ giấy này cho người khác, họ có thể tìm đến đúng ngôi nhà đó để xem hoặc sửa sang đồ đạc bên trong.

Dưới đây là đoạn code minh họa cơ bản:

```
#include <stdio.h>

int main() {
    int x = 10; // Biến thông thường
    int *ptr;   // Khai báo con trỏ ptr kiểu int

    ptr = &x;   // Gán địa chỉ của x cho ptr

    printf("Giá trị của x: %d\n", x);
    printf("Địa chỉ của x: %p\n", &x);
    printf("Giá trị ptr đang lưu trữ (địa chỉ của x): %p\n", ptr);
    printf("Giá trị tại địa chỉ mà ptr trỏ đến: %d\n", *ptr);

    return 0;
}
```

Nâng cao

"Sử dụng ẩn dụ 'địa chỉ nhà' để giải thích con trỏ. Thực hiện theo 3 bước (Chain-of-Thought): 1. Khai báo, 2. Gán địa chỉ, 3. Truy xuất. Nhấn mạnh lỗi Segmentation Fault."

Hãy tưởng tượng bộ nhớ máy tính là một dãy phố dài với rất nhiều ngôi nhà, mỗi ngôi nhà có một **số nhà** duy nhất (địa chỉ) và bên trong chứa **đồ đạc** (dữ liệu). Con trỏ chính là một "tờ giấy ghi chú" dùng để chép lại số nhà đó.

Để làm chủ con trỏ, chúng ta thực hiện theo quy trình 3 bước logic sau:

- 1. Khai báo (Tạo tờ giấy ghi chú)**
Đầu tiên, bạn cần một tờ giấy để ghi địa chỉ. Trong C, khi bạn viết `int *ptr;`, bạn đang nói với máy tính: "Hãy cho tôi một biến tên là `ptr`, nhưng dùng nó để chứa đồ đạc, hãy dùng nó để chứa **số nhà** của một ngôi nhà kiểu `int` nào đó". Dấu `*` chính là ký hiệu xác nhận đây là một "tờ giấy ghi địa chỉ" chứ không phải bản thân ngôi nhà.
- 2. Gán địa chỉ (Chép số nhà vào giấy)**
Giả sử bạn có một ngôi nhà tên là `x` đang chứa giá trị `10`. Để lấy số nhà của `x`, ta dùng toán tử `&`. Lệnh `ptr = &x;` hành động giống như việc bạn nhìn lên biển số nhà của `x` và chép chính xác con số đó vào tờ giấy `ptr`. Lúc này, tờ giấy `ptr` đang "trỏ" thẳng đến ngôi nhà `x`.
- 3. Truy xuất (Tìm đến nhà và tác động vào đồ đạc)**
Khi bạn muốn xem hoặc thay đổi đồ đạc bên trong ngôi nhà mà bạn đã ghi địa chỉ trên giấy, bạn dùng toán tử giải tham chiếu `*`. Khi viết `*ptr = 20;`, máy tính sẽ đọc địa chỉ trên tờ giấy `ptr`, đi đến đúng ngôi nhà đó và thay thế đồ đạc cũ bằng giá trị `20`. Mọi thay đổi qua `*ptr` thực chất là thay đổi trực tiếp lên biến `x`.

Nâng cao

Cảnh báo: Lỗi Segmentation Fault (Địa chỉ "ma")

Đây là lỗi kinh điển nhất mà các sinh viên kỹ thuật máy tính thường gặp. Hãy tưởng tượng bạn có một tờ giấy ghi chú nhưng **chưa ghi địa chỉ nào** (con trỏ hoang - wild pointer) hoặc ghi một địa chỉ của khu vực quản sự cấm vào (vùng nhớ của hệ điều hành).

Khi bạn cố gắng dùng lệnh `*ptr` để xông vào "ngôi nhà" không tồn tại hoặc bị cấm đó, hệ thống sẽ ngay lập tức ngăn chặn để bảo vệ an toàn cho toàn bộ khu phố. Kết quả là chương trình của bạn bị "văng" ra ngoài và thông báo lỗi **Segmentation Fault** (Lỗi phân đoạn bộ nhớ). Điều này nhắc nhở bạn luôn phải đảm bảo con trỏ đã được trỏ vào một "ngôi nhà" hợp lệ trước khi muốn thay đổi nội dung bên trong nó.

Hình 2: Sử dụng Chain-of-Thought để giải mã rõ hơn về “Con trỏ trong ngôn ngữ C”.

3. Tác vụ 3: Tạo bộ câu hỏi ôn tập môn Giải tích (Chuỗi Fourier)

Cấp độ	Nội dung Prompt	Sắc thái kết quả quan sát
Cơ bản	"Tạo 5 câu hỏi về chuỗi Fourier."	Câu hỏi thường chỉ dừng ở mức độ nhận biết công thức, quá dễ hoặc lặp lại.
Cải tiến	"Tạo 5 câu hỏi về chuỗi Fourier kèm đáp án và lời giải chi tiết cho các hệ số a_n , b_n ."	Bổ sung phần lời giải giúp người học tự đối chiếu và sửa lỗi sai trong tính toán.
Nâng cao	"Đóng vai trợ giảng Toán lý. Dựa trên thang đo Bloom, tạo 5 câu hỏi: 2 câu nhận biết, 2 câu vận dụng tính toán, 1 câu ứng dụng xử lý tín hiệu. Đảm bảo đáp án nhiều có tính logic."	Thang đo Bloom đảm bảo độ phủ kiến thức. Việc yêu cầu đáp án nhiều logic giúp rèn luyện khả năng phân biệt các lỗi sai thường gặp khi làm bài thi thật.

Cơ bản

"Tạo 5 câu hỏi về chuỗi Fourier."

Dưới đây là 5 câu hỏi tự luận cơ bản để bạn nắm vững lý thuyết trọng tâm của chuỗi Fourier:

- Định nghĩa:** Viết công thức tổng quát của chuỗi Fourier cho một hàm số $f(x)$ tuần hoàn với chu kỳ 2π trên đoạn $[-\pi, \pi]$. Giải thích ý nghĩa của các hệ số a_n, b_n .
- Tính đối xứng:** Chứng minh rằng nếu $f(x)$ là một hàm chẵn trên đoạn $[-L, L]$, thì tất cả các hệ số b_n đều bằng 0. Khi đó, chuỗi Fourier được gọi là chuỗi gì?
- Điều kiện hội tụ:** Trình bày nội dung định lý Dirichlet về điều kiện để một hàm số có thể khai triển thành chuỗi Fourier. Chuỗi này hội tụ về giá trị nào tại các điểm gián đoạn?
- Hàm số cụ thể:** Nêu các bước để tìm chuỗi Fourier cho hàm số $f(x) = x$ trên khoảng $(-\pi, \pi)$. Tại sao ta có thể khẳng định ngay $a_n = 0$ mà không cần tính tích phân?
- Ứng dụng tần số:** Trong khai triển Fourier, số hạng ứng với $n = 1$ được gọi là gì? Mối liên hệ giữa chỉ số n và tần số của các sóng thành phần là như thế nào?

Cải tiến

"Tạo 5 câu hỏi về chuỗi Fourier kèm đáp án và lời giải chi tiết cho các hệ số a_n, b_n nhé."

1. Cho hàm số $f(x) = x$ tuần hoàn với chu kỳ 2π trên khoảng $(-\pi, \pi)$. Hãy xác định giá trị của hệ số Fourier a_n với mọi $n \geq 0$.

• **Đáp án:** $a_n = 0$.

• **Lời giải chi tiết:** Vì $f(x) = x$ là một hàm lẻ trên khoảng đối xứng $(-\pi, \pi)$, tích của nó với hàm $\cos(nx)$ (hàm chẵn) sẽ là một hàm lẻ. Tích phân của một hàm lẻ trên đoạn đối xứng $[-\pi, \pi]$ luôn bằng 0. Do đó, tất cả các hệ số a_n (bao gồm cả a_0) đều triệt tiêu.

2. Với hàm số $f(x) = |x|$ tuần hoàn chu kỳ 2π trên đoạn $[-\pi, \pi]$, hệ số nào sau đây chắc chắn bằng 0?

• **Đáp án:** $b_n = 0$ với mọi $n \geq 1$.

• **Lời giải chi tiết:** Hàm $f(x) = |x|$ là hàm chẵn ($f(-x) = |-x| = |x| = f(x)$). Trong khai triển Fourier, hàm chẵn chỉ chứa các thành phần cosin và hằng số a_0 . Tích của hàm chẵn $|x|$ và hàm lẻ $\sin(nx)$ là một hàm lẻ, nên tích phân $b_n = \frac{1}{\pi} \int_{-\pi}^{\pi} |x| \sin(nx) dx = 0$.

3. Tính hệ số a_0 cho hàm số $f(x) = 1$ khi $0 < x < \pi$ và $f(x) = 0$ khi $-\pi < x < 0$, với chu kỳ $T = 2\pi$.

• **Đáp án:** $a_0 = 1$.

• **Lời giải chi tiết:** Áp dụng công thức $a_0 = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(x) dx$. Ta chia khoảng tích phân: $a_0 = \frac{1}{2\pi} \left(\int_{-\pi}^0 0 dx + \int_0^{\pi} 1 dx \right) = \frac{1}{2\pi} (0 + \pi) = \frac{1}{2}$.

Nâng cao

"Đóng vai trợ giảng Toán lý. Dựa trên thang đo Bloom, tạo 5 câu hỏi: 2 câu nhận biết, 2 câu vận dụng tính toán, 1 câu ứng dụng xử lý tín hiệu. Đảm bảo đáp án nhiều có tính logic."

Chào bạn, mình là trợ giảng Toán lý của bạn đây. Dựa trên yêu cầu và các cấp độ tư duy trong thang đo Bloom, mình đã thiết kế bộ 5 câu hỏi về **Chuỗi Fourier** với các phương án nhiễu được xây dựng dựa trên những sai sót phổ biến của sinh viên (như nhầm lẫn tính chẵn lẻ, sai hệ số chuẩn hóa hoặc nhầm công thức tích phân).

Dưới đây là nội dung chi tiết:

- (Nhận biết)** Trong khai triển chuỗi Fourier của hàm số $f(x)$ tuần hoàn chu kỳ $2L$ trên đoạn $[-L, L]$, hệ số a_0 được xác định bởi công thức nào sau đây?
 - A. $a_0 = \frac{1}{2L} \int_{-L}^L f(x) dx$ (Nhiều: Nhầm với công thức tính giá trị trung bình).
 - B. $a_0 = \frac{1}{L} \int_{-L}^L f(x) dx$ (Đáp án đúng).
 - C. $a_0 = \frac{1}{2} \int_{-L}^L f(x) dx$ (Nhiều: Nhầm hệ số của chu kỳ 2π).
 - D. $a_0 = \int_{-L}^L f(x) dx$ (Nhiều: Thiếu hệ số chuẩn hóa).
- (Nhận biết)** Đối với một hàm số lẻ $f(x)$ trên khoảng đối xứng $[-L, L]$, thành phần nào sau đây chắc chắn bằng 0 trong chuỗi Fourier?
 - A. Tất cả các hệ số a_n với $n \geq 0$ (Đáp án đúng: Do tính chất trực giao của hàm lẻ và hàm cos).
 - B. Tất cả các hệ số b_n với $n \geq 1$ (Nhiều: Đây là đặc điểm của hàm chẵn).
- (Vận dụng)** Tính hệ số b_1 cho hàm số $f(x) = 2$ khi $0 < x < \pi$ và $f(x) = -2$ khi $-\pi < x < 0$ (chu kỳ 2π).
 - A. $b_1 = 0$ (Nhiều: Nhầm đây là hàm chẵn).
 - B. $b_1 = \frac{2}{\pi}$ (Nhiều: Quên nhân đôi khi tính tích phân trên nửa chu kỳ).
 - C. $b_1 = \frac{4}{\pi}$ (Đáp án đúng: $\frac{2}{\pi} \int_0^{\pi} 2 \sin(x) dx = \frac{4}{\pi} [-\cos(x)]_0^{\pi} = \frac{4}{\pi}$).
 - D. $b_1 = \frac{2}{\pi}$ (Nhiều: Sai sót trong bước thế cận tích phân).
- (Vận dụng)** Xác định hệ số a_0 của hàm số $f(x) = x^2$ trên đoạn $[-\pi, \pi]$ (chu kỳ 2π).
 - A. $a_0 = \frac{\pi^2}{3}$ (Nhiều: Đây là giá trị $a_0/2$).
 - B. $a_0 = \frac{2\pi^2}{3}$ (Đáp án đúng: $\frac{1}{2\pi} \int_{-\pi}^{\pi} x^2 dx = \frac{2\pi^2}{3}$).
 - C. $a_0 = 0$ (Nhiều: Nhầm x^2 là hàm lẻ).
 - D. $a_0 = \pi^2$ (Nhiều: Tính sai nguyên hàm hoặc thiếu hệ chia 3).

Hình 3: So sánh các cấp độ câu hỏi ôn tập từ mức độ Nhận biết đến Vận dụng.

III. PHÂN TÍCH CHUYÊN SÂU VÀ TƯ DUY PHẢN BIỆN

Tại sao cùng một mô hình AI nhưng lại cho ra các kết quả với "trí tuệ" khác nhau? Có sự liên hệ mật thiết giữa cấu trúc Prompt và hiệu quả trả lời:

- Sức mạnh của Role-play (Gán vai):** Việc gán vai không chỉ là hình thức mà là thao tác "khoanh vùng tri thức". Khi đóng vai một "Kỹ sư máy tính" hay "Giảng viên UET", AI sẽ kích hoạt các lớp ngữ nghĩa chuyên biệt, ưu tiên sử dụng thuật ngữ kỹ thuật chính xác và các tiêu chuẩn học thuật. Điều này loại bỏ hoàn toàn các câu trả lời mang tính "phổ thông" và tăng độ tin cậy cho các tác vụ khó như giải mã cấu trúc RISC-V hay gỡ lỗi con trỏ trong C.
- Chain-of-Thought (Chuỗi tư duy) - Kiểm soát logic:** Prompt cơ bản thường khiến AI dự đoán kết quả một cách nhảy vọt, dễ dẫn đến sai lầm trong tính toán Giải tích. Khi yêu cầu "suy nghĩ từng bước", chúng ta tạo ra các điểm neo logic trung gian. Quá trình này buộc AI phải tự kiểm chứng dữ liệu ở từng bước thực hiện, giúp kiểm soát chất lượng đầu ra, tăng tính xác thực và giảm thiểu tối đa hiện tượng "ảo giác AI" (hallucination).
- Sự chuyển dịch từ "Thông tin" sang "Tri thức":** Prompt nâng cao giúp AI thực hiện quá trình **Tổng hợp (Synthesis)** thay vì chỉ liệt kê. Thay vì chỉ lặp lại định nghĩa về Chuỗi Fourier, AI được hướng dẫn để kết nối lý thuyết với ứng dụng thực tế (như xử lý tín hiệu) và tạo ra các mẹo ghi nhớ hoặc ví dụ ẩn dụ (như địa chỉ nhà cho con

trở). Điều này biến dữ liệu thô thành tri thức hữu dụng, hỗ trợ trực tiếp cho quá trình tiếp thu của người học.

4. **Thiết lập "Hành lang dữ liệu" (Constraints):** Việc sử dụng các ràng buộc về định dạng (như yêu cầu lập bảng, giới hạn số chữ) đóng vai trò là bộ lọc thông tin nhiễu. Trong học tập kỹ thuật, việc loại bỏ các diễn đạt rườm rà và tập trung vào các thông số cốt lõi giúp sinh viên tiết kiệm thời gian hệ thống hóa kiến thức, đồng thời rèn luyện tư duy trình bày vấn đề một cách khoa học và chuyên nghiệp.

IV. TỔNG HỢP NGUYÊN TẮC VIẾT PROMPT SÁNG TẠO (L.O.G.I.C)

Tôi đúc kết bộ nguyên tắc **L.O.G.I.C** để tận dụng tối đa AI trong học tập:

- **L - Layering (Phân tầng):** Đưa yêu cầu từ tổng quát đến định dạng cụ thể (Bảng, mục lục).
- **O - Objective Role (Vai trò định hướng):** Xác định danh tính để AI điều chỉnh tông giọng chuyên biệt.
- **G - Guided Reasoning (Dẫn dắt suy luận):** Luôn yêu cầu "Giải thích tại sao" hoặc "Tư duy từng bước".
- **I - Iterative Refinement (Tinh chỉnh lặp lại):** Phản hồi dựa trên kết quả đầu tiên để yêu cầu sâu hơn.
- **C - Constraint & Format (Ràng buộc định dạng):** Sử dụng các mốc giới hạn (độ dài, đối tượng đọc) để tránh lan man.

V. KẾT LUẬN

Chất lượng của câu trả lời tỷ lệ thuận với sự thấu đáo trong câu hỏi. **Prompt Engineering** không chỉ là kỹ thuật, mà là cách chúng ta cấu trúc lại tư duy của chính mình để giao tiếp hiệu quả với trí tuệ nhân tạo.